UNIWERSYTET WSB MERITO W WARSZAWIE

Imię i nazwisko: Vasyl Tryndiuk

Numer albumu: 159856

Tytuł pracy dyplomowej

Projekt i implementacja warstwy front-endowej aplikacji webowej do rezerwacji seansów kinowych z wykorzystaniem biblioteki React

Praca inżynierska

Promotor:
dr inż. Piotr Bobiński

Kierunek: Informatyka
Specjalność: Front-end Developer

Warszawa, 2026

SPIS TREŚCI

WSTĘP 3
ROZDZIAŁ 1. Analiza problemu i wymagań systemu rezerwacji 5
1.1. Charakterystyka problemu rezerwacji miejsc w kinie 5
1.2. Proces rezerwacji miejsc w aplikacji 7
1.3. Funkcjonalności systemu 10
1.4. Logowanie użytkownika do systemu 11
1.5. Wymagania niefunkcjonalne systemu 13
ROZDZIAŁ 2. Projekt systemu informatycznego 14
2.1. Architektura systemu 14
2.2. Baza danych systemu 16
2.3. Model danych systemu 19
2.4. Diagramy UML systemu 20
ROZDZIAŁ 3. Implementacja aplikacji 23
3.1. Logika działania aplikacji – wybór miejsc i interakcja użytkownika 23
3.2. Komunikacja z backendem i przetwarzanie danych 26
3.3. Autoryzacja i obsługa użytkownika 32
3.4. Testowanie aplikacji 36
ZAKOŃCZENIE 41
BIBLIOGRAFIA 42
AKTY NORMATYWNE 43

WSTĘP

Współczesne usługi internetowe coraz częściej wykorzystują systemy umożliwiające użytkownikom samodzielne wykonywanie różnych operacji online. Jednym z przykładów takich rozwiązań są systemy rezerwacyjne stosowane między innymi w kinach, teatrach, hotelach czy transporcie. Dzięki nim użytkownik może sprawdzić dostępność miejsc, wybrać odpowiedni termin oraz dokonać rezerwacji bez konieczności osobistego kontaktu z obsługą.

W przypadku kin systemy rezerwacji miejsc odgrywają szczególnie ważną rolę. Pozwalają one użytkownikowi zaplanować wizytę na wybrany seans filmowy oraz wybrać konkretne miejsca na sali kinowej. Dla użytkownika oznacza to większą wygodę oraz oszczędność czasu. Z punktu widzenia organizacji kina system taki pozwala na lepsze zarządzanie dostępnością miejsc oraz automatyzację procesu rezerwacji.

Projektowanie systemów rezerwacyjnych wiąże się jednak z pewnymi wyzwaniami technicznymi. Jednym z najważniejszych problemów jest zapewnienie poprawnej synchronizacji danych. System musi zapobiegać sytuacji, w której dwóch użytkowników próbuje zarezerwować to samo miejsce w tym samym czasie. Wymaga to odpowiedniego zaprojektowania komunikacji między warstwą front-end aplikacji, serwerem oraz bazą danych.

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji webowej umożliwiającej rezerwację miejsc na seanse kinowe. System pozwala użytkownikowi przeglądać listę filmów, wybierać dostępne godziny seansów oraz dokonywać rezerwacji konkretnych miejsc na sali kinowej. Zrealizowane rozwiązanie zostało oparte na architekturze klient-serwer.

Warstwa front-end aplikacji została wykonana z wykorzystaniem biblioteki React[1]. Odpowiada ona za prezentację danych oraz interakcję z użytkownikiem. Warstwa serwerowa została zaimplementowana w technologii Spring Boot[2], która umożliwia tworzenie aplikacji backendowych w języku Java. Komunikacja pomiędzy front-endem i backendem odbywa się przy użyciu interfejsu REST API[3]. Dane o filmach, seansach oraz rezerwacjach przechowywane są w relacyjnej bazie danych MySQL[4].

Przedmiotem badań w niniejszej pracy jest aplikacja webowa do rezerwacji miejsc na seanse kinowe oraz sposób integracji jej warstwy front-end z backendowym interfejsem API i bazą danych. Analizie podlega architektura systemu, sposób zapisu rezerwacji w bazie danych oraz logika wyboru miejsc przez użytkownika.

---

[1] React – Official documentation,
https://react.dev
[2] Spring Boot – Official documentation,
https://spring.io/projects/spring-boot
[3] Fielding R., REST API architectural style,
https://restfulapi.net/
[4] MySQL Reference Manual,
https://dev.mysql.com/doc/

Struktura pracy została podzielona na trzy główne rozdziały. W pierwszym rozdziale przedstawiono analizę problemu rezerwacji miejsc w kinie oraz wymagania funkcjonalne i niefunkcjonalne systemu. W drugim rozdziale opisano projekt systemu informatycznego, w tym architekturę aplikacji, strukturę bazy danych oraz diagramy UML. Trzeci rozdział poświęcony jest implementacji aplikacji oraz testowaniu działania systemu w środowisku lokalnym.

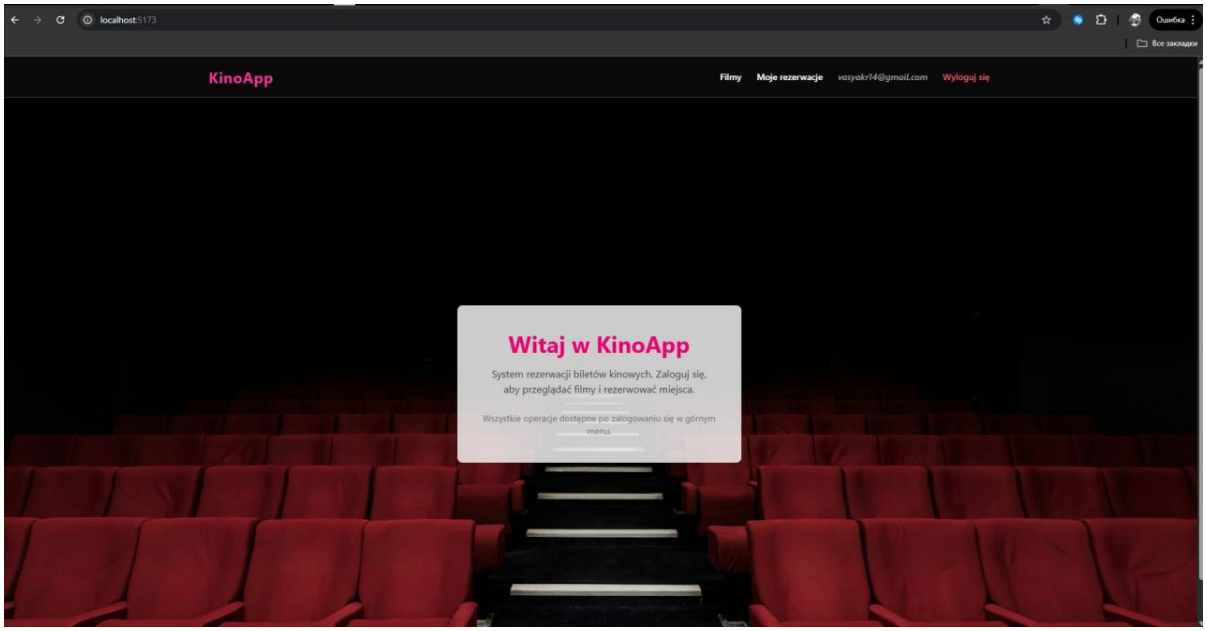
ROZDZIAŁ 1. Analiza problemu i wymagań systemu rezerwacji

1.1. Charakterystyka problemu rezerwacji miejsc w kinie

W przypadku kina użytkownik chce zazwyczaj sprawdzić repertuar filmowy, wybrać interesujący go seans oraz zarezerwować odpowiednie miejsca na sali kinowej. Tradycyjny sposób rezerwacji biletów polega na zakupie w kasie kina, co może wiązać się z koniecznością stania w kolejce oraz ograniczoną dostępnością miejsc.

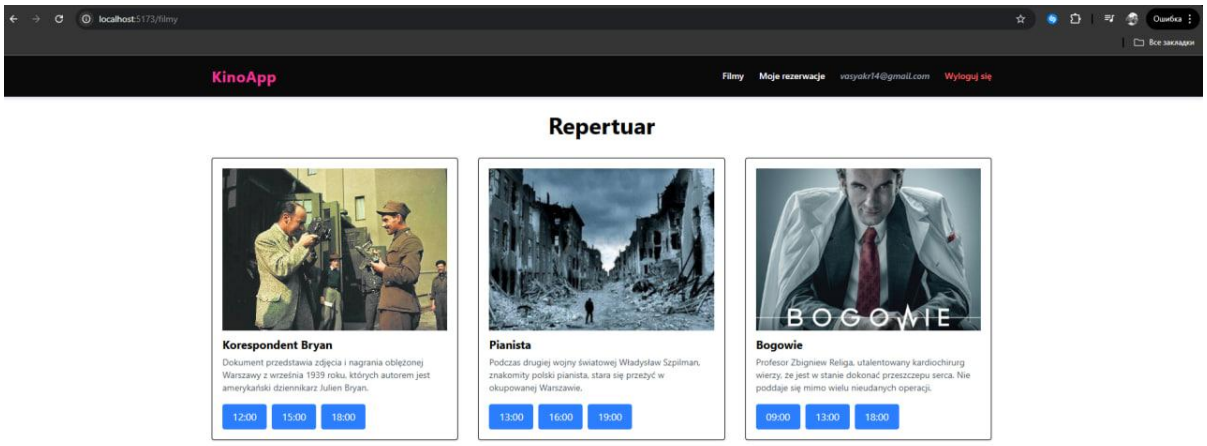
Zastosowanie aplikacji webowej umożliwia wykonanie tych operacji w prosty i szybki sposób za pomocą przeglądarki internetowej.

Po uruchomieniu aplikacji użytkownik widzi stronę główną zawierającą nagłówek oraz podstawowe informacje powitalne.



Rysunek 1.1 Strona główna aplikacji
Źródło: opracowanie własne

Strona główna pełni funkcję informacyjną i zawiera elementy nawigacyjne umożliwiające przejście do dalszych części aplikacji.
Lista filmów dostępnych w repertuarze wyświetlana jest na osobnej stronie aplikacji. Użytkownik może tam zobaczyć dostępne filmy oraz godziny seansów.



Rysunek 1.2 Lista filmów wraz z dostępnymi godzinami seansów
Źródło: opracowanie własne

W tym widoku użytkownik może wybrać konkretną godzinę seansu dla danego filmu. Po wybraniu godziny aplikacja przenosi użytkownika bezpośrednio do strony rezerwacji miejsc.

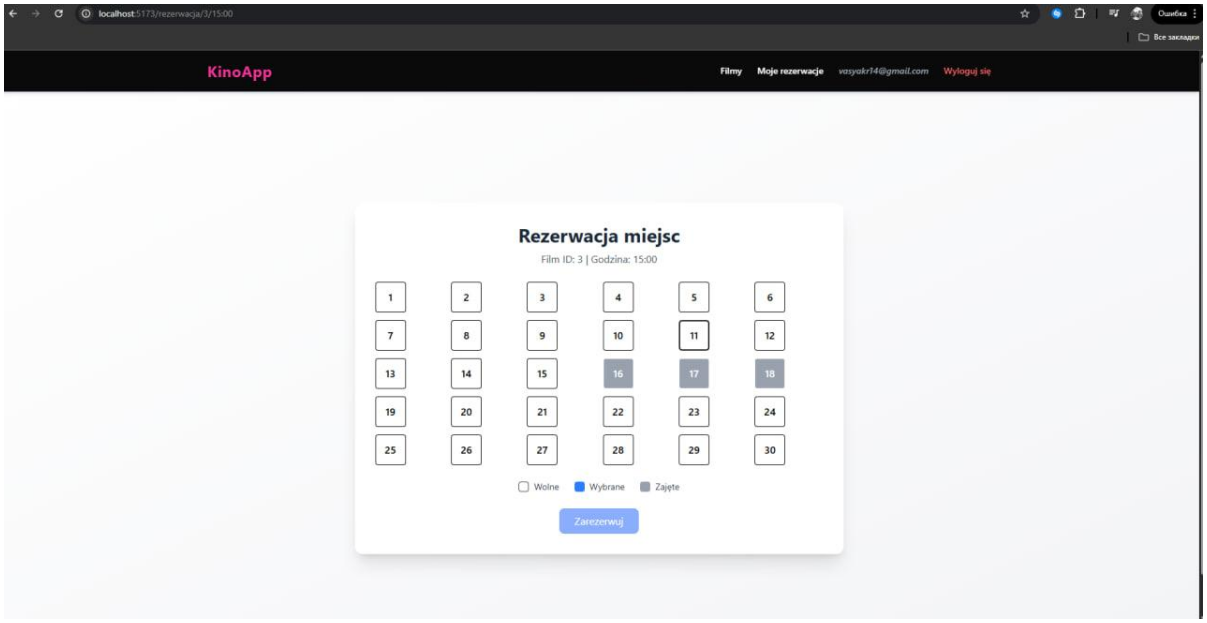
W przeciwieństwie do niektórych systemów rezerwacyjnych aplikacja nie posiada oddzielnej strony szczegółów filmu. Wszystkie informacje o filmie oraz dostępne godziny seansów są prezentowane bezpośrednio na stronie listy filmów.

Rysunek 1.3 Wybór godziny seansu dla wybranego filmu
Źródło: opracowanie własne

1.2. Proces rezerwacji miejsc w aplikacji

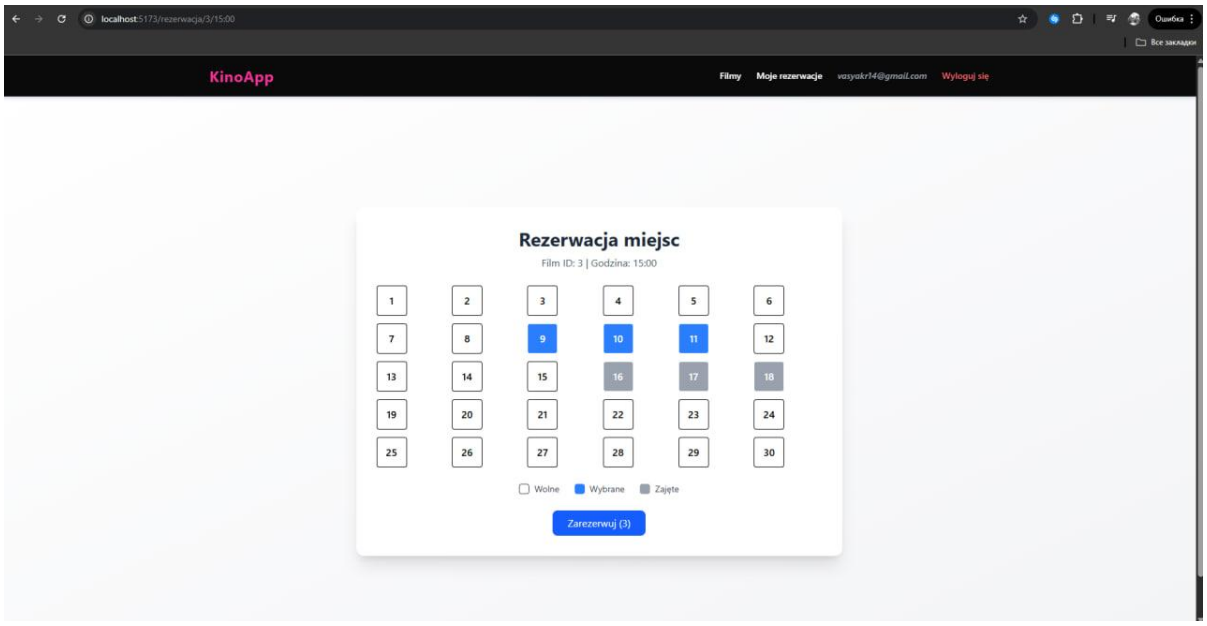
Po wybraniu godziny seansu użytkownik zostaje przeniesiony do widoku rezerwacji miejsc. Na tej stronie wyświetlana jest siatka miejsc dostępnych na sali kinowej. Każde miejsce posiada swój numer oraz aktualny stan dostępności.

System generuje siatkę miejsc w sposób dynamiczny. W prezentowanej aplikacji sala kinowa zawiera trzydzieści miejsc. Każde z nich może znajdować się w jednym z trzech stanów: wolne, wybrane przez użytkownika lub zajęte.



Rysunek 1.4 Widok siatki miejsc przed dokonaniem wyboru
Źródło: opracowanie własne

W przypadku gdy dane miejsce zostało już zarezerwowane przez innego użytkownika, system oznacza je jako zajęte i uniemożliwia jego ponowny wybór. Dzięki temu użytkownik widzi aktualną dostępność miejsc na wybrany seansu.

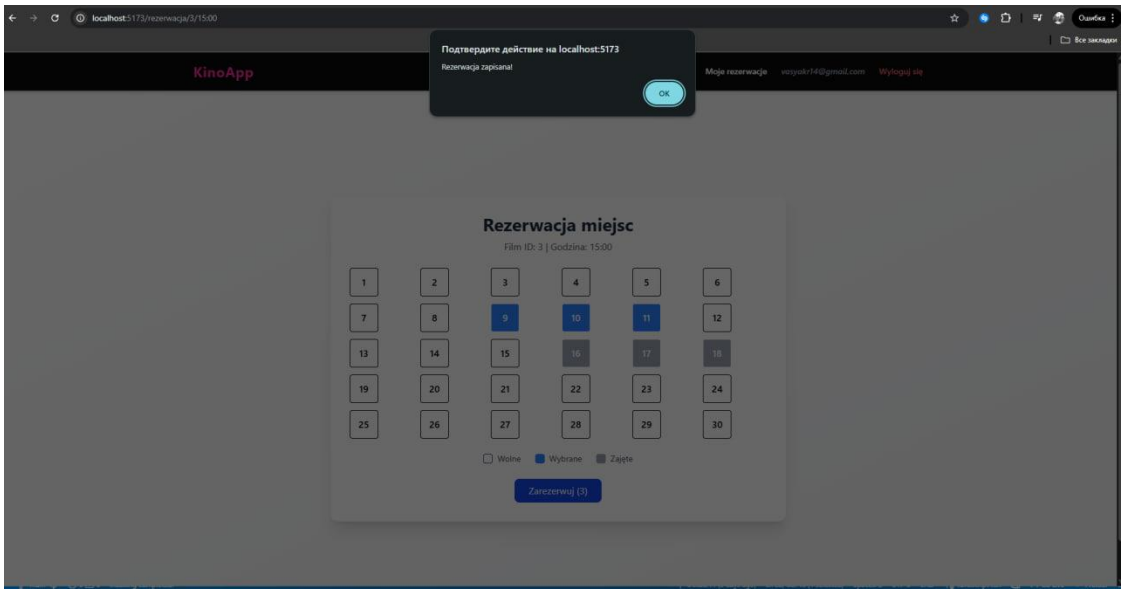


Rysunek 1.5 Widok wybranych miejsc przez użytkownika
Źródło: opracowanie własne

Wybrane miejsca są oznaczone innym kolorem, co ułatwia użytkownikowi kontrolę nad dokonywaną rezerwacją. System umożliwia wybór jednego lub kilku miejsc jednocześnie.

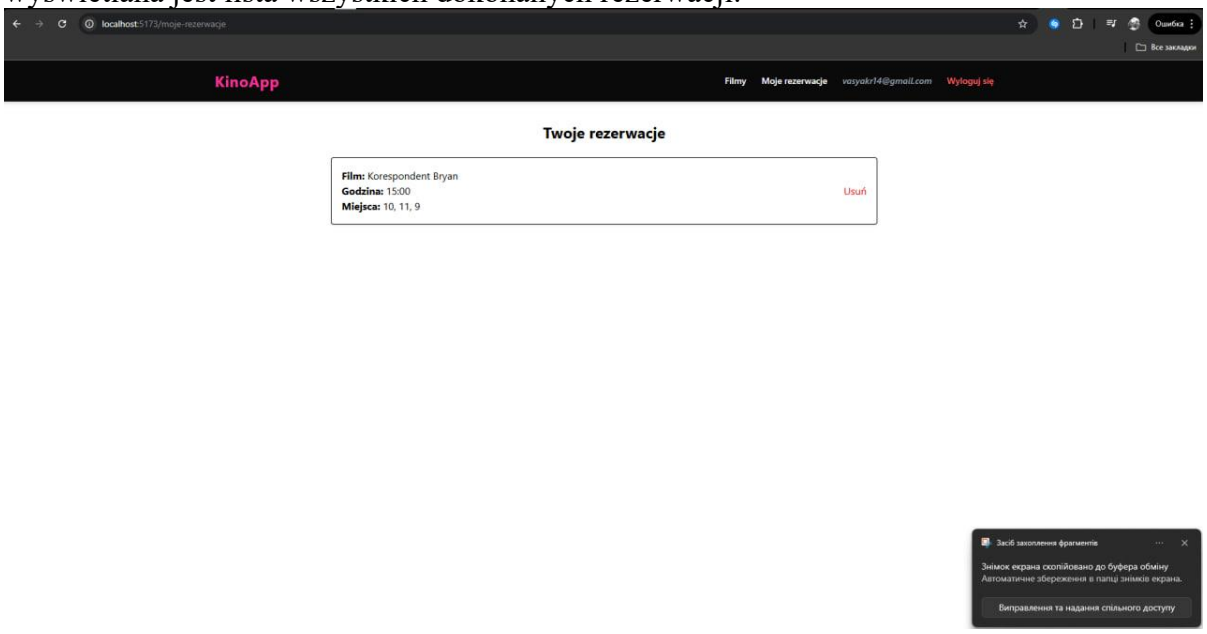
Po wybraniu miejsc użytkownik może zapisać rezerwację w systemie poprzez kliknięcie przycisku rezerwacji. W tym momencie aplikacja wysyła dane do serwera, gdzie są one zapisywane w bazie danych.

Po poprawnym zapisaniu rezerwacji użytkownik otrzymuje komunikat potwierdzający wykonanie operacji.



Rysunek 1.6 Komunikat potwierdzający zapisanie rezerwacji
Źródło: opracowanie własne

Po zapisaniu rezerwacji użytkownik może przejść do sekcji swoich rezerwacji, w której wyświetlana jest lista wszystkich dokonanych rezerwacji.



Rysunek 1.7 Lista rezerwacji użytkownika
Źródło: opracowanie własne

1.3. Funkcjonalności systemu

Na podstawie analizy działania aplikacji można określić główne funkcjonalności systemu rezerwacji miejsc.

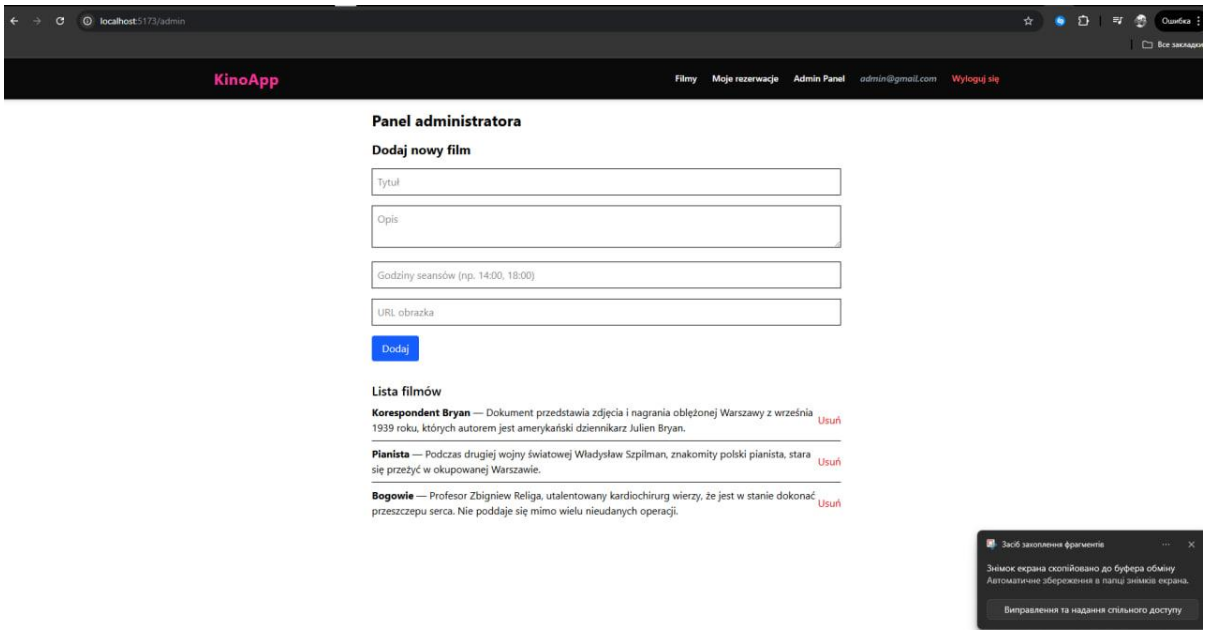
Do najważniejszych funkcji systemu należą:

1. wyświetlanie listy filmów dostępnych w repertuarze,
2. wybór godziny seansu,
3. wyświetlanie siatki miejsc na sali kinowej,
4. oznaczanie miejsc już zajętych przez innych użytkowników,
5. możliwość wyboru jednego lub kilku miejsc,
6. zapisanie rezerwacji w bazie danych,
7. wyświetlanie listy rezerwacji użytkownika,
8. usuwanie istniejącej rezerwacji.

System zawiera również panel administracyjny umożliwiający zarządzanie repertuarem filmowym.

Administrator może:

- dodawać nowe filmy,
- usuwać filmy.

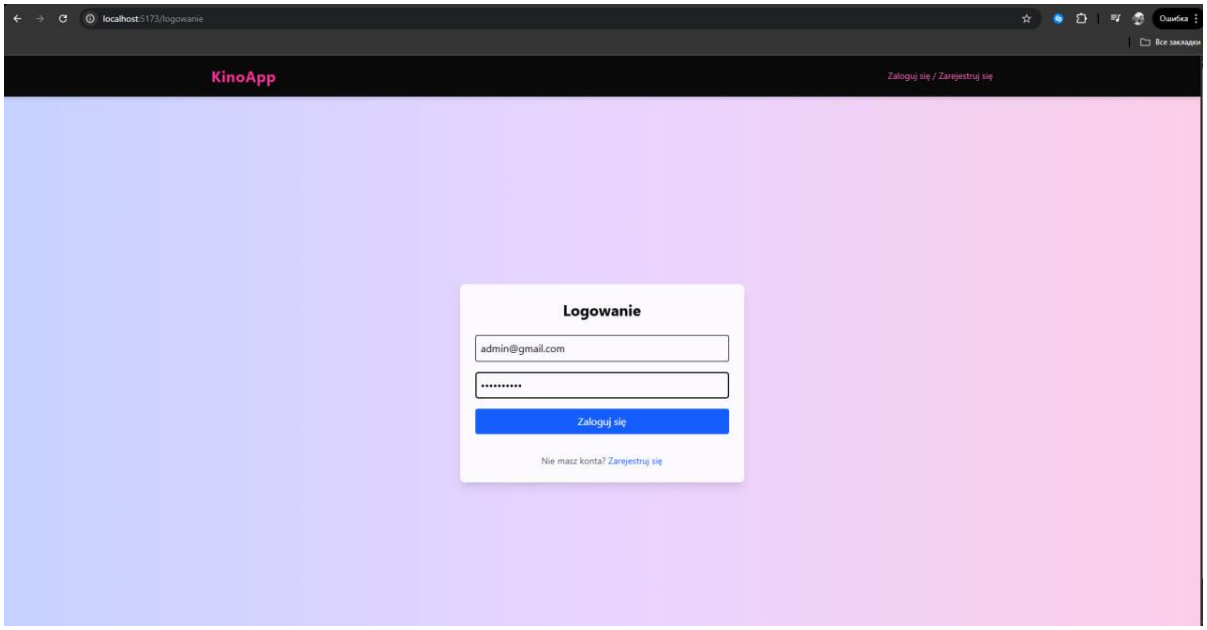


Rysunek 1.8 Panel administracyjny aplikacji
Źródło: opracowanie własne

1.4. Logowanie użytkownika do systemu

Przed skorzystaniem z funkcjonalności systemu użytkownik musi zalogować się do aplikacji. Proces logowania umożliwia identyfikację użytkownika oraz przypisanie rezerwacji do jego konta.

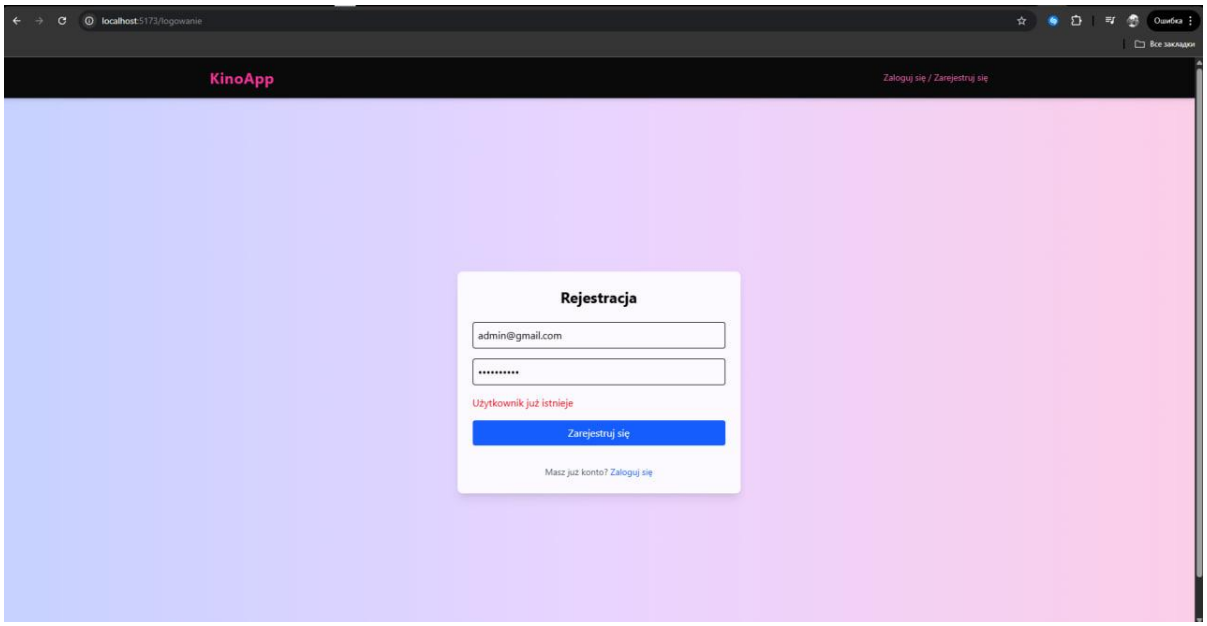
Na stronie logowania użytkownik wprowadza adres e-mail oraz hasło. Dane te są następnie przesyłane do serwera, gdzie następuje ich weryfikacja. W przypadku poprawnych danych użytkownik zostaje zalogowany do systemu i uzyskuje dostęp do funkcjonalności aplikacji.



Rysunek 1.9 Strona logowania użytkownika
Źródło: opracowanie własne

Po wprowadzeniu poprawnych danych użytkownik zostaje przekierowany do głównej części aplikacji, gdzie może przeglądać dostępne filmy oraz dokonywać rezerwacji miejsc.

W przypadku podania niepoprawnych danych logowania system informuje użytkownika o błędzie.



Rysunek 1.10 Formularz rejestracji użytkownika
Źródło: opracowanie własne

1.5. Wymagania niefunkcjonalne systemu

Oprócz funkcjonalności system musi spełniać również wymagania niefunkcjonalne. Dotyczą one jakości działania aplikacji oraz komfortu korzystania z systemu przez użytkownika. Jednym z najważniejszych wymagań jest czytelny i intuicyjny interfejs użytkownika.

Aplikacja powinna umożliwiać szybkie znalezienie filmu, wybór seansu oraz dokonanie rezerwacji miejsc bez konieczności wykonywania skomplikowanych operacji.

Istotnym aspektem działania systemu jest również poprawna synchronizacja danych. W systemach rezerwacyjnych szczególnie ważne jest zapobieganie sytuacji, w której dwóch użytkowników próbuje zarezerwować to samo miejsce. W prezentowanej aplikacji przed wyświetleniem siatki miejsc system pobiera z serwera listę zajętych miejsc dla wybranego seansu.

Kolejnym wymaganiem jest szybka komunikacja pomiędzy aplikacją front-end a serwerem. Dzięki zastosowaniu architektury klient-serwer oraz interfejsu REST API dane mogą być przesyłane w sposób efektywny i szybki.

Ważnym elementem jest również przechowywanie danych w bazie danych w sposób umożliwiający ich późniejsze odczytanie oraz analizę. Informacje o rezerwacjach zapisywane są w relacyjnej bazie danych, co pozwala na ich łatwe wyszukiwanie oraz zarządzanie nimi.

ROZDZIAŁ 2. Projekt systemu informatycznego

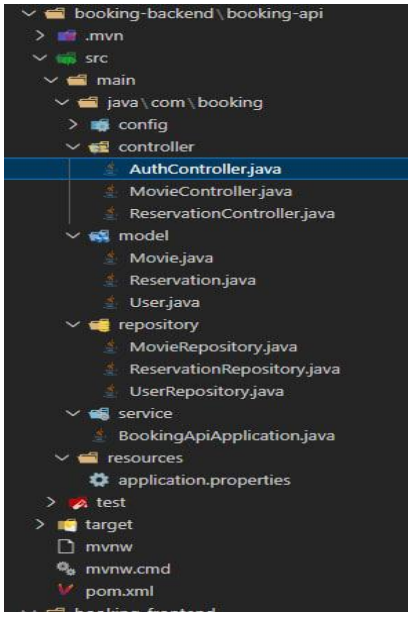
2.1. Architektura systemu

Front-end aplikacji został wykonany z wykorzystaniem biblioteki React. Odpowiada on za wyświetlanie danych użytkownikowi oraz obsługę interakcji takich jak wybór filmu, godziny seansu oraz miejsc na sali kinowej.

Back-end aplikacji został zaimplementowany w technologii Spring Boot. Jego zadaniem jest obsługa żądań przychodzących z aplikacji front-end, przetwarzanie danych oraz zapis informacji o rezerwacjach w bazie danych.

Komunikacja pomiędzy front-endem i back-endem odbywa się przy użyciu interfejsu REST API oraz formatu danych JSON[5].

Dzięki takiemu podziałowi aplikacja jest łatwiejsza w rozwijaniu oraz utrzymaniu, ponieważ warstwa prezentacji jest oddzielona od logiki systemu oraz bazy danych.



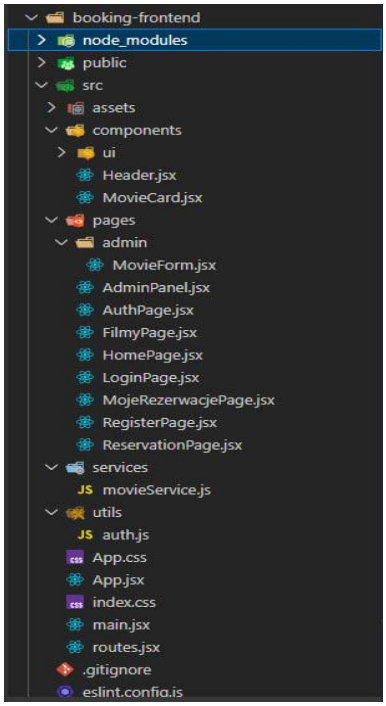
Rysunek 2.1 Struktura projektu backend
Źródło: opracowanie własne

Backend aplikacji składa się z kilku podstawowych elementów. Najważniejsze z nich to kontrolery, modele danych oraz repozytoria odpowiedzialne za komunikację z bazą danych.

Kontrolery obsługują żądania HTTP przychodzące z aplikacji front-end. Modele danych

[5] JSON format specification, https://www.json.org

reprezentują strukturę danych przechowywanych w bazie danych, natomiast repozytoria umożliwiają wykonywanie operacji na danych takich jak zapis, odczyt czy usuwanie rekordów.



Rysunek 2.2 Struktura projektu front-end
Źródło: opracowanie własne

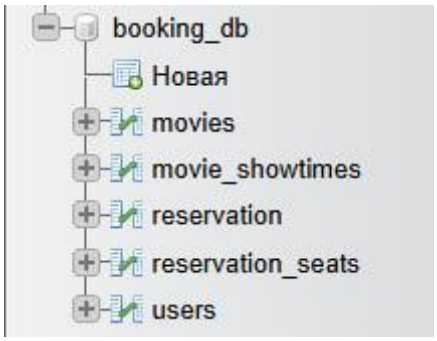
Front-end aplikacji został podzielony na komponenty odpowiedzialne za różne części interfejsu użytkownika. Każdy komponent odpowiada za konkretną funkcjonalność, na przykład wyświetlanie listy filmów, obsługę rezerwacji miejsc lub wyświetlanie listy rezerwacji użytkownika.

Takie podejście pozwala na łatwiejsze zarządzanie kodem aplikacji oraz umożliwia ponowne wykorzystanie komponentów w różnych częściach systemu.

2.2. Baza danych systemu

System rezerwacji wykorzystuje relacyjną bazę danych MySQL do przechowywania informacji o filmach, seansach oraz rezerwacjach użytkowników.

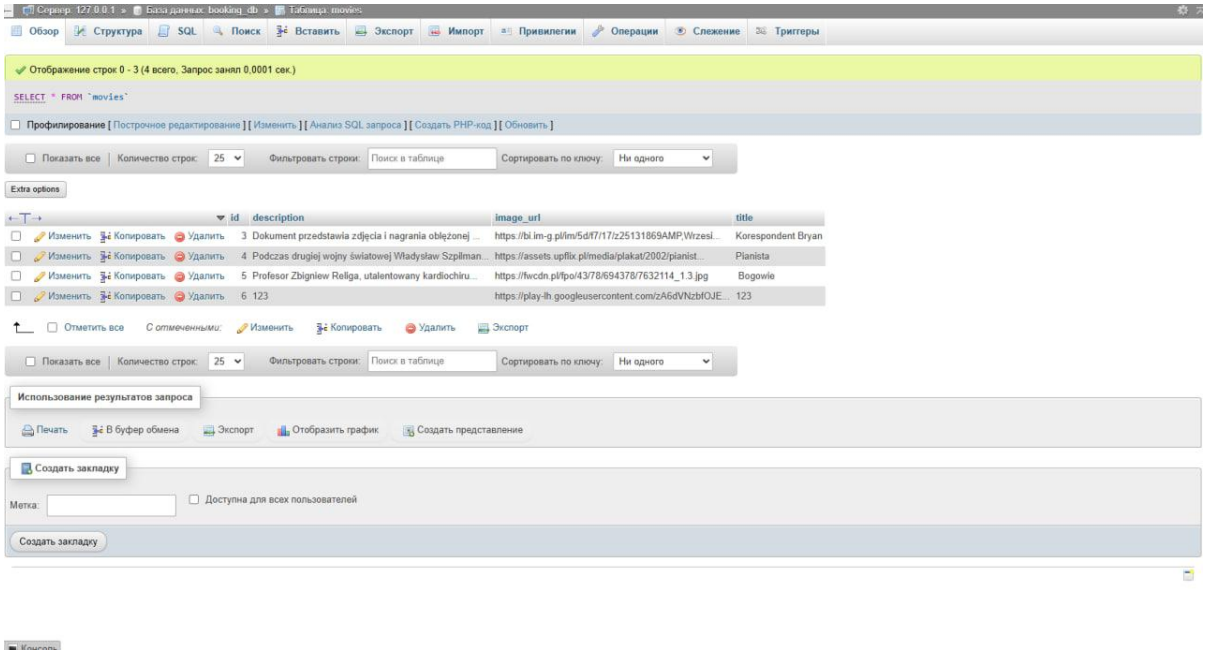
Baza danych została zaprojektowana w taki sposób, aby umożliwić przechowywanie wszystkich informacji potrzebnych do obsługi procesu rezerwacji miejsc.



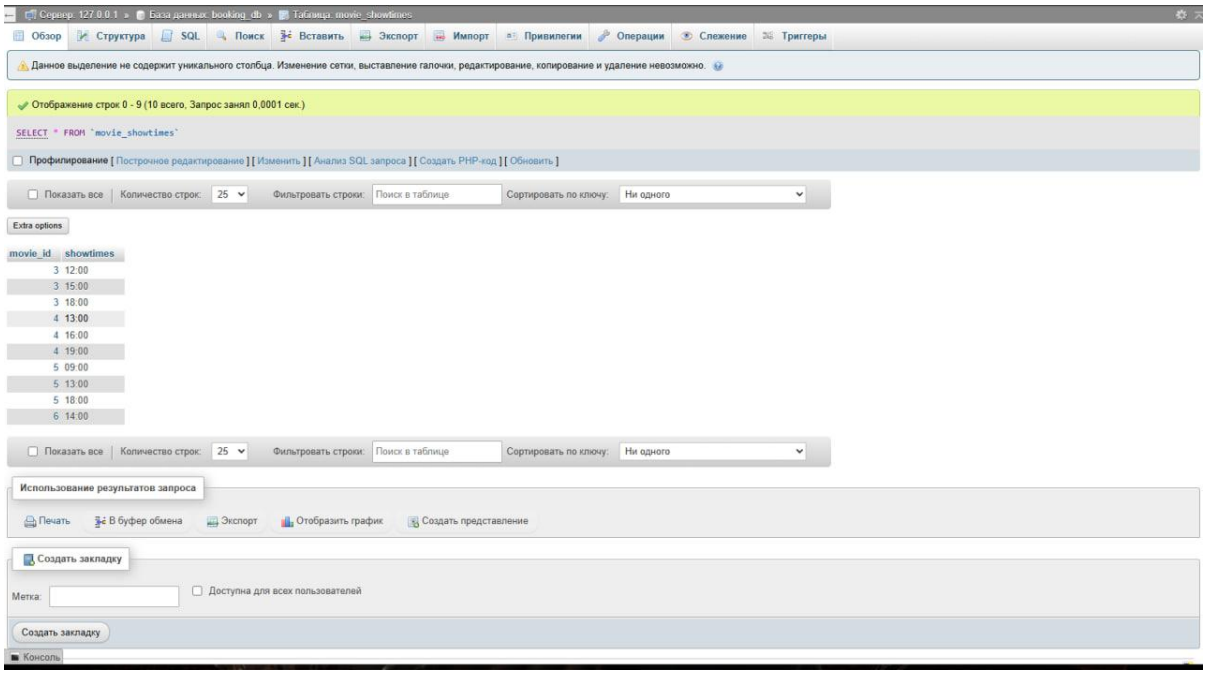
Rysunek 2.3 Struktura tabel w bazie danych
Źródło: opracowanie własne

Najważniejsze tabele w bazie danych to:

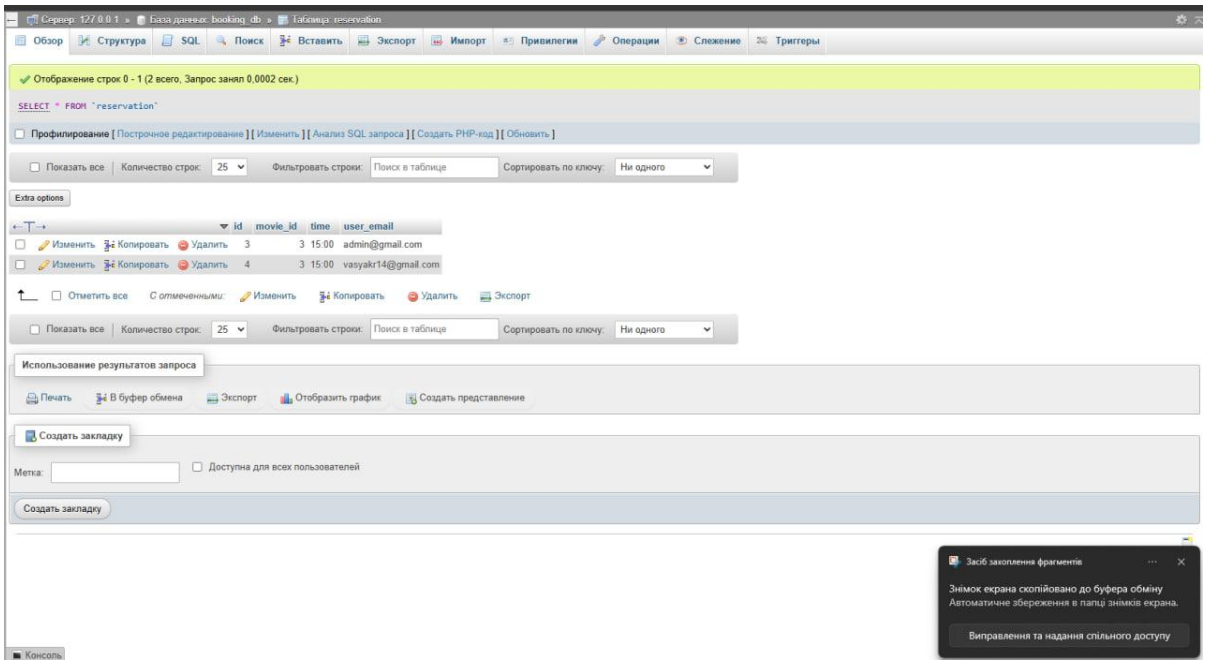
- **movies** – przechowuje informacje o filmach dostępnych w repertuarze,
- **movies_showtimes** – zawiera godziny seansów dla poszczególnych filmów,
- **users** – przechowuje dane użytkowników systemu,
- **reservation** – zawiera informacje o dokonanych rezerwacjach,
- **reservation_seats** – przechowuje numery miejsc przypisanych do danej rezerwacji.



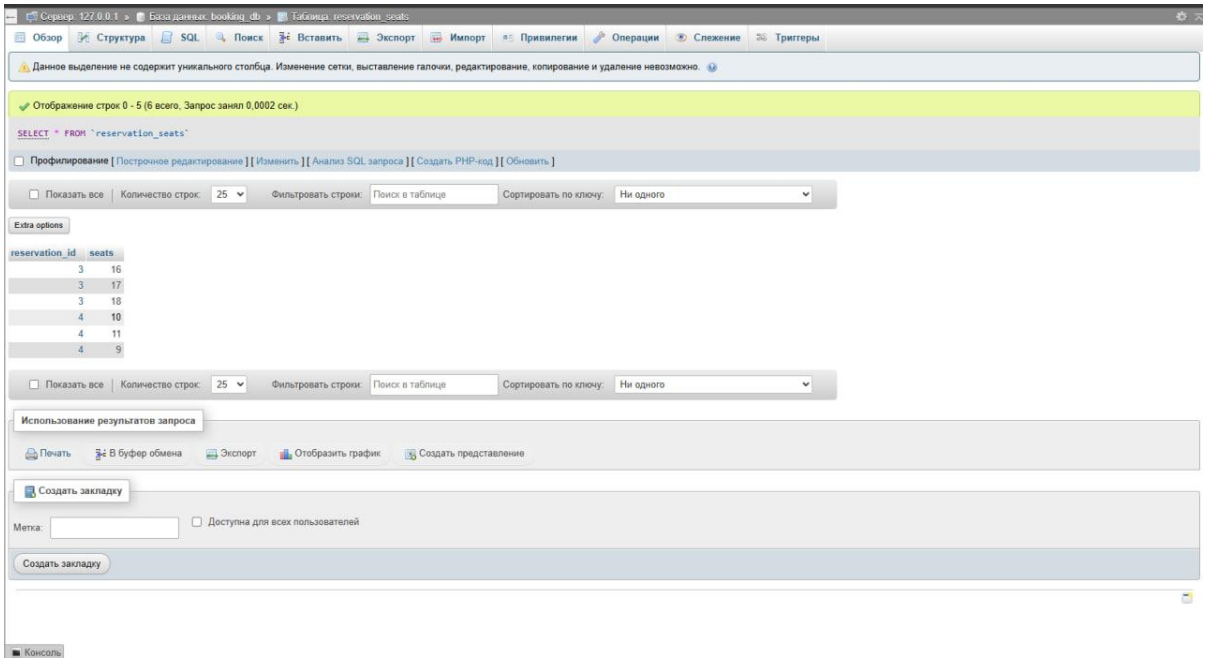
Rysunek 2.4 Tabela movies w bazie danych
Źródło: opracowanie własne



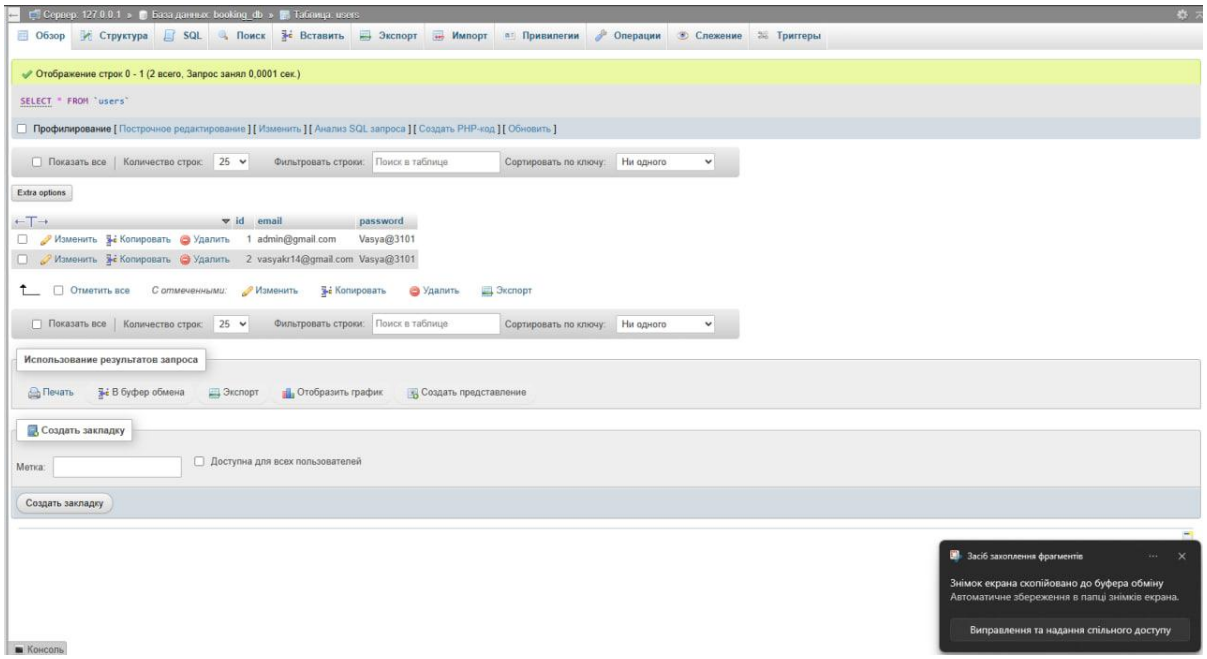
Rysunek 2.5 Tabela seansów filmowych
Źródło: opracowanie własne



Rysunek 2.6 Tabela rezerwacji
Źródło: opracowanie własne



Rysunek 2.7 Tabela miejsc przypisanych do rezerwacji
Źródło: opracowanie własne



Rysunek 2.8 Tabela użytkowników
Źródło: opracowanie własne

2.3. Model danych systemu

W systemie rezerwacji seansów kinowych dane przechowywane są w relacyjnej bazie danych. Model danych został zaprojektowany w taki sposób, aby umożliwić przechowywanie informacji o filmach, seansach, użytkownikach oraz dokonanych rezerwacjach.

Najważniejsze elementy modelu danych to:

- film,
- seans,
- użytkownik,
- rezerwacja.

Każdy z tych elementów reprezentowany jest przez osobną encję w systemie oraz odpowiadającą jej tabelę w bazie danych.

Encja **Movie** przechowuje podstawowe informacje o filmie, takie jak tytuł oraz dostępne godziny seansów. Dane te wykorzystywane są do wyświetlania repertuaru w aplikacji.

Encja **Reservation** reprezentuje pojedynczą rezerwację dokonaną przez użytkownika. Zawiera ona informacje o wybranym filmie, godzinie seansu, adresie e-mail użytkownika oraz liście wybranych miejsc.

Encja **User** przechowuje dane użytkownika systemu, które są wykorzystywane podczas procesu logowania oraz przypisywania rezerwacji do konkretnego konta.

Zaprojektowany model danych umożliwia łatwe wyszukiwanie rezerwacji dla konkretnego użytkownika, pobieranie zajętych miejsc dla wybranego seansu oraz zarządzanie repertuarem filmowym.

2.4. Diagramy UML systemu

W celu lepszego przedstawienia działania systemu przygotowano diagramy UML. Diagramy pozwalają w graficzny sposób zaprezentować funkcjonalności systemu oraz przepływ danych pomiędzy jego komponentami.
W pracy wykorzystano dwa typy diagramów UML:

- diagram przypadków użycia,
- diagram sekwencji.

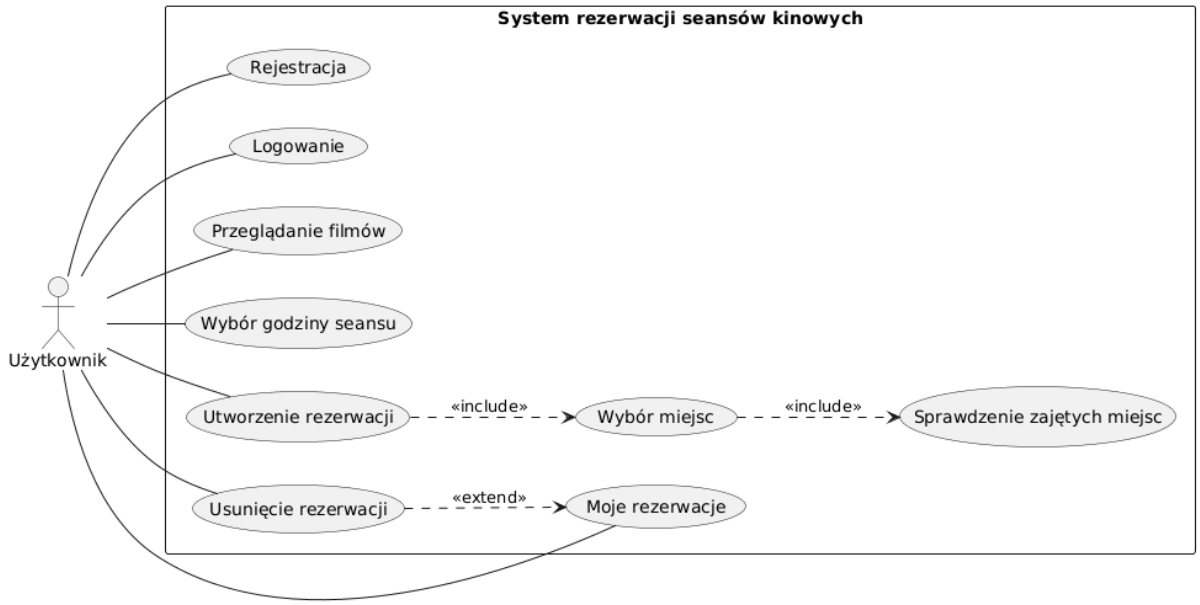
Diagram przypadków użycia

Diagram przypadków użycia przedstawia funkcjonalności systemu z punktu widzenia użytkownika. Głównym aktorem systemu jest użytkownik korzystający z aplikacji rezerwacji seansów kinowych.
Użytkownik może wykonywać następujące operacje:

- rejestracja konta,
- logowanie do systemu,
- przeglądanie dostępnych filmów,
- wybór godziny seansu,
- utworzenie rezerwacji,
- przeglądanie własnych rezerwacji,
- usuwanie istniejącej rezerwacji.

Proces rezerwacji składa się z kilku kroków. W celu utworzenia rezerwacji użytkownik musi najpierw wybrać miejsca na sali kinowej. System przed wyświetleniem siatki miejsc sprawdza, które miejsca są już zajęte przez innych użytkowników.

Diagram przedstawia zależności pomiędzy poszczególnymi funkcjonalnościami systemu oraz logiczny przebieg procesu rezerwacji.



Rysunek 2.9 Diagram przypadków użycia systemu rezerwacji
Źródło: opracowanie własne

Diagram sekwencji procesu rezerwacji

Diagram sekwencji przedstawia przebieg komunikacji pomiędzy poszczególnymi elementami systemu podczas procesu tworzenia rezerwacji.

W procesie uczestniczą następujące komponenty:

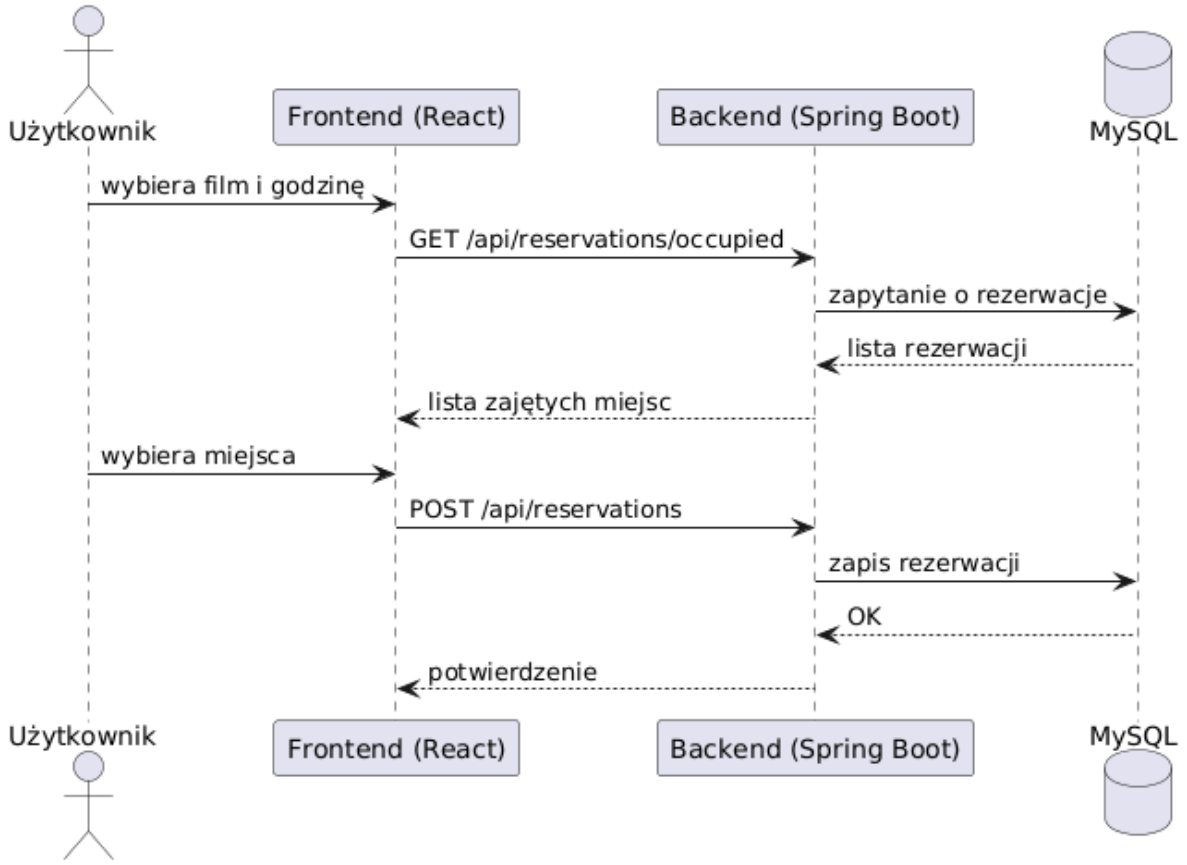
- użytkownik,
- warstwa front-end aplikacji,
- backend aplikacji,
- baza danych MySQL.

Proces rozpoczyna się od wyboru filmu oraz godziny seansu przez użytkownika. Następnie aplikacja front-end wysyła zapytanie HTTP do backendu w celu pobrania informacji o zajętych miejscach.

Backend pobiera dane z bazy danych i zwraca listę zajętych miejsc do aplikacji front-end. Po wyświetleniu dostępnych miejsc użytkownik może wybrać miejsca na sali kinowej.

Następnie front-end wysyła zapytanie POST do endpointu /api/reservations, które zawiera dane rezerwacji. Backend zapisuje dane w bazie danych, a następnie zwraca informację o poprawnym wykonaniu operacji.

Diagram sekwencji przedstawia pełny przepływ danych pomiędzy warstwą prezentacji, logiką biznesową oraz bazą danych.



Rysunek 2.10 Diagram sekwencji procesu rezerwacji
Źródło: opracowanie własne

ROZDZIAŁ 3. Implementacja aplikacji

3.1. Logika działania aplikacji – wybór miejsc i interakcja użytkownika

Jednym z najważniejszych elementów aplikacji jest mechanizm wyboru miejsc na sali kinowej. Funkcja ta odpowiada za interakcję użytkownika z systemem podczas procesu rezerwacji seansu.

Logika wyboru miejsc została zaimplementowana w komponencie odpowiedzialnym za obsługę rezerwacji. Mechanizm ten wykorzystuje stan aplikacji React do przechowywania informacji o wybranych oraz zajętych miejscach.

Obsługa kliknięcia w miejsce została zaimplementowana w funkcji przedstawionej w listingu 3.1.

```
const handleSeatClick = (seat) => {
  if (selectedSeats.includes(seat)) {
    setSelectedSeats(selectedSeats.filter(s => s !== seat));
  } else {
    setSelectedSeats([...selectedSeats, seat]);
  }
};
```

Listing 3.1 Obsługa wyboru miejsc w React

Funkcja sprawdza, czy dane miejsce znajduje się już na liście miejsc wybranych przez użytkownika.
Jeśli tak, zostaje ono usunięte z listy. W przeciwnym przypadku miejsce zostaje dodane do listy wybranych miejsc.

Dzięki wykorzystaniu mechanizmu stanu komponentu interfejs użytkownika jest automatycznie aktualizowany po każdej zmianie.

Proces wyboru miejsc
Proces rezerwacji miejsc w aplikacji przebiega w kilku etapach:

1. użytkownik wybiera film z listy repertuaru,
2. użytkownik wybiera godzinę seansu,
3. aplikacja pobiera z backendu listę zajętych miejsc,
4. wyświetlana jest siatka miejsc na sali kinowej,
5. użytkownik wybiera wolne miejsca,
6. system zapisuje rezerwację w bazie danych.

Każdy z tych etapów kontrolowany jest przez stan aplikacji React.

Generowanie siatki miejsc
W prezentowanej wersji aplikacji sala zawiera **30 miejsc**.
Miejsca są generowane dynamicznie w pętli, co oznacza, że nie są zapisane ręcznie w

kodzie. Frontend generuje listę numerów miejsc i renderuje je jako elementy interfejsu użytkownika.
Takie podejście umożliwia łatwe dostosowanie liczby miejsc w przyszłości oraz upraszcza strukturę kodu.

Stany aplikacji

W logice wyboru miejsc wykorzystano dwa podstawowe stany aplikacji:

- **selectedSeats** – lista miejsc wybranych przez użytkownika,
- **occupiedSeats** – lista miejsc zajętych, pobrana z backendu.

Dzięki rozdzieleniu tych dwóch list aplikacja może w prosty sposób określić, które miejsca są dostępne, które zostały wybrane przez użytkownika, a które są już zajęte.

Podczas renderowania interfejsu każde miejsce sprawdza, czy znajduje się na jednej z tych list. Na tej podstawie zmieniany jest jego wygląd w interfejsie użytkownika.

Synchronizacja danych o zajętych miejscach
Po wybraniu filmu oraz godziny seansu aplikacja pobiera aktualną listę zajętych miejsc.
Dane te są wykorzystywane do oznaczenia niedostępnych miejsc w interfejsie użytkownika.

W tym celu wykonywane jest zapytanie do endpointu:
GET /api/reservations/occupied

Zapytanie zawiera parametry identyfikujące wybrany seans:
• movieId
• time
Odpowiedź serwera zawiera listę numerów zajętych miejsc. Lista ta zapisywana jest w stanie aplikacji occupiedSeats.

Na tej podstawie aplikacja może:

- blokować wybór miejsc już zajętych,
- wyświetlać aktualny stan dostępności miejsc,
- zapobiegać wyborowi miejsc zarezerwowanych przez innych użytkowników.

Aktualizacja interfejsu użytkownika
React automatycznie aktualizuje interfejs po każdej zmianie stanu komponentu.
Oznacza to, że po:

- dodaniu miejsca do listy selectedSeats,
- usunięciu miejsca z listy,
- pobraniu listy occupiedSeats z backendu

widok aplikacji jest natychmiast aktualizowany.

Użytkownik od razu widzi zmiany w interfejsie bez konieczności ręcznego odświeżania strony.
Walidacja danych przed zapisaniem rezerwacji

Przed wysłaniem danych do backendu aplikacja wykonuje podstawową walidację danych wejściowych.
Sprawdzane są między innymi następujące warunki:

- czy użytkownik wybrał godzinę seansu,
- czy wybrano przynajmniej jedno miejsce.

Przykładowa walidacja została przedstawiona w listingu 3.2.

```
if (!time) {
alert("Wybierz godzinę seansu.");
return;
}

if (selectedSeats.length === 0) {
alert("Wybierz miejsca.");
return;
}
```

Listing 3.2 Walidacja wyboru miejsc

Takie rozwiązanie zapobiega wysyłaniu niepoprawnych danych do backendu.

Możliwe problemy i ich rozwiązanie

Podczas projektowania logiki wyboru miejsc uwzględniono również potencjalne problemy związane z działaniem systemu.

Jednym z nich jest sytuacja, w której dwóch użytkowników próbuje zarezerwować to samo miejsce w tym samym czasie. W takim przypadku backend zapisuje pierwszą poprawną rezerwację, natomiast przy kolejnej próbie system może ponownie pobrać listę zajętych miejsc.

Innym możliwym problemem jest utrata połączenia z serwerem. W takiej sytuacji aplikacja informuje użytkownika o błędzie i umożliwia ponowienie operacji.

3.2. Komunikacja z backendem i przetwarzanie danych

Komunikacja pomiędzy warstwą front-end a backendem była zaimplementowana przy użyciu zapytań HTTP kierowanych do interfejsu REST API. Frontend pobiera dane potrzebne do wyświetlenia repertuaru, sprawdzenia zajętości miejsc, zapisania rezerwacji oraz pobrania listy rezerwacji użytkownika. Backend przetwarza otrzymane dane i wykonuje operacje na bazie danych.

W systemie wykorzystano trzy główne grupy operacji:

- operacje związane z repertuarem filmowym,
- operacje związane z rezerwacjami,
- operacje związane z obsługą użytkownika.

Pobieranie repertuaru filmowego

Lista filmów prezentowana użytkownikowi pobierana jest z backendu za pomocą endpointu GET /api/movies. Backend zwraca listę obiektów reprezentujących filmy zapisane w bazie danych. W aktualnej wersji projektu metoda pobierająca repertuar została zaimplementowana w kontrolerze filmów.

```
@GetMapping
public List<Movie> getAllMovies() {
return movieRepository.findAll();
}
```

Listing 3.3 Endpoint zwracający listę filmów

Metoda zwraca wszystkie rekordy zapisane w tabeli filmów. Po stronie front-end dane te są wykorzystywane do wygenerowania listy kart filmów, zawierających podstawowe informacje o tytule, opisie i godzinach seansów. Przykładowa odpowiedź serwera ma postać obiektów JSON zawierających pola id, title, description, showtimes oraz imageUrl.

Pobieranie zajętych miejsc dla wybranego seansu

W poprzednim podrozdziale opisano sposób wykorzystania danych o zajętych miejscach po stronie interfejsu użytkownika. W tej części przedstawiono implementację operacji backendu odpowiedzialnych za obsługę zapytań aplikacji.

Po wybraniu filmu oraz godziny seansu frontend musi pobrać aktualną listę zajętych miejsc. Ta część systemu została zaimplementowana w komponencie ReservationPage.jsx. Po wejściu na stronę rezerwacji wykonywane jest zapytanie do endpointu GET /api/reservations/occupied, zawierające parametry movieId oraz time.

```
useEffect(() => {
fetch(`http://localhost:8080/api/reservations/occupied?movieId=${movieId}&time=${time}`
)
   .then((res) => res.json())
   .then((data) => {
     setOccupiedSeats(data);
   });
}, [movieId, time]);
```

Listing 3.4 Pobieranie zajętych miejsc w komponencie rezerwacji

Po stronie front-end odpowiedź serwera jest zapisywana w stanie occupiedSeats. Na tej podstawie system oznacza miejsca jako zajęte i blokuje możliwość ich ponownego wyboru. Dzięki temu użytkownik widzi aktualny stan sali dla wybranego seansu.

Po stronie backendu obsługa tego endpointu została zrealizowana w kontrolerze rezerwacji.

```
@GetMapping("/occupied")
public ResponseEntity<List<Integer>> getOccupiedSeats(
      @RequestParam int movieId,
      @RequestParam String time
) {
   List<Reservation> reservations =
reservationRepository.findByMovieIdAndTime(movieId, time);
   List<Integer> occupiedSeats = reservations.stream()
         .flatMap(r -> r.getSeats().stream())
         .collect(Collectors.toList());
   return ResponseEntity.ok(occupiedSeats);
}
```

Listing 3.5 Pobieranie listy zajętych miejsc w backendzie

Metoda pobiera z bazy wszystkie rezerwacje przypisane do wskazanego filmu i godziny seansu, a następnie łączy listy miejsc w jedną kolekcję numerów miejsc zajętych. To właśnie ta lista zwracana jest do front-endu. Dzięki temu aplikacja nie musi samodzielnie analizować danych rezerwacji, ponieważ backend przygotowuje już gotową odpowiedź.

Generowanie i zapis nowej rezerwacji

Po wyborze miejsc użytkownik może zapisać rezerwację w systemie. W tym celu frontend tworzy obiekt zawierający wszystkie dane potrzebne do zapisu: identyfikator filmu, godzinę seansu, listę wybranych miejsc oraz adres e-mail użytkownika.

```
const reservation = {
  movieId: parseInt(movieId),
  time,
  seats: selectedSeats,
  userEmail: localStorage.getItem("email") || "test@example.com",
};
```

Listing 3.6 Tworzenie obiektu rezerwacji po stronie front-end

W przedstawionej implementacji adres e-mail pobierany jest z pamięci przeglądarki. Takie rozwiązanie pozwala powiązać rezerwację z konkretnym użytkownikiem bez konieczności ponownego wpisywania danych przy każdej operacji.

Po utworzeniu obiektu frontend wysyła zapytanie HTTP typu POST do backendu.

```
fetch("http://localhost:8080/api/reservations", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(reservation),
})
  .then((res) => res.json())
  .then(() => {
    alert("Rezerwacja zapisana!");
    setSelectedSeats([]);
    navigate("/moje-rezerwacje");
  })
  .catch(() => alert("Błąd podczas rezerwacji."));
```

Listing 3.7 Wysyłanie rezerwacji do backendu

Po poprawnym zapisaniu danych użytkownik otrzymuje komunikat potwierdzający, a następnie zostaje przekierowany do widoku „Moje rezerwacje”. W przypadku błędu system wyświetla odpowiedni komunikat.

Po stronie backendu zapis nowej rezerwacji obsługiwany jest przez metodę createReservation.

```
@PostMapping
public ResponseEntity<Reservation> createReservation(@RequestBody Reservation
reservation) {
    Reservation saved = reservationRepository.save(reservation);
    return ResponseEntity.ok(saved);
}
```

Listing 3.8 Zapis nowej rezerwacji w backendzie

Metoda przyjmuje obiekt Reservation przesłany w formacie JSON, zapisuje go za pomocą repozytorium JPA[6] i zwraca odpowiedź z obiektem zapisanej rezerwacji. Z punktu widzenia front-endu jest to końcowy etap procesu rezerwacji.

Struktura obiektu rezerwacji została zdefiniowana w encji Reservation.

```
@Entity
public class Reservation {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private int movieId;
   private String time;
   private String userEmail;

   @ElementCollection
   private List<Integer> seats;
}
```

Listing 3.9 Struktura danych rezerwacji

Encja zawiera wszystkie dane potrzebne do odtworzenia rezerwacji w systemie: powiązanie z filmem, godziną, użytkownikiem oraz listą wybranych miejsc. Szczególnie istotne jest pole seats, które umożliwia zapisanie wielu miejsc w ramach jednej rezerwacji.

Pobieranie rezerwacji użytkownika

Po zapisaniu rezerwacji użytkownik może przejść do widoku „Moje rezerwacje”, w którym prezentowana jest lista wszystkich rezerwacji przypisanych do jego adresu e-mail. W tym celu backend udostępnia endpoint GET /api/reservations/user/{email}. Operacja ta została zaimplementowana w kontrolerze rezerwacji.

```
@GetMapping("/user/{email}")
public List<Reservation> getReservationsByUserEmail(@PathVariable String email) {
   return reservationRepository.findByUserEmail(email);
}
```

Listing 3.10 Pobieranie rezerwacji użytkownika

Metoda wykorzystuje repozytorium JPA do wyszukania wszystkich rezerwacji przypisanych do konkretnego użytkownika. Dzięki temu możliwe jest wyświetlanie historii rezerwacji bez konieczności pobierania wszystkich danych z systemu.

Usuwanie istniejącej rezerwacji

[6] Spring Data JPA documentation
https://spring.io/projects/spring-data-jpa

System umożliwia również usunięcie wcześniej zapisanej rezerwacji. Funkcja ta jest istotna z punktu widzenia użytkownika, ponieważ pozwala zarządzać zapisanymi wcześniej miejscami.
Operacja usuwania została zrealizowana w backendzie przez endpoint DELETE /api/reservations/{id}.

```
@DeleteMapping("/{id}")
public ResponseEntity<?> deleteReservation(@PathVariable Long id) {
   if (!reservationRepository.existsById(id)) {
      return ResponseEntity.badRequest().body("Reservation not found");
   }
   reservationRepository.deleteById(id);
   return ResponseEntity.ok().build();
}
```

Listing 3.11 Usuwanie rezerwacji

Przed usunięciem backend sprawdza, czy rezerwacja o podanym identyfikatorze istnieje w bazie danych. Jeśli nie, zwracana jest odpowiedź błędu. W przeciwnym przypadku rekord zostaje usunięty, a klient otrzymuje potwierdzenie poprawnego wykonania operacji.

Operacje administracyjne związane z repertuarem

W projekcie zaimplementowano także podstawowe funkcje administracyjne związane z zarządzaniem repertuarem. Administrator może dodać nowy film oraz usunąć istniejący rekord. W aktualnym dokumencie znajduje się fragment metody dodawania filmu.

```
@PostMapping
public Movie addMovie(@RequestBody Movie movie) {
   return movieRepository.save(movie);
}
```

Listing 3.12 Dodawanie filmu w backendzie

Metoda przyjmuje dane filmu w żądaniu HTTP i zapisuje je w bazie danych. Dzięki temu nowy film może zostać później wyświetlony na liście repertuaru. Implementacja tego mechanizmu pozwala zarządzać zawartością aplikacji bez konieczności ręcznej modyfikacji danych po stronie bazy.

Zestawienie wykorzystanych endpointów

W tabeli 3.1 przedstawiono zestawienie najważniejszych endpointów wykorzystanych w aplikacji.

Tabela 3.1 Najważniejsze endpointy systemu

- GET /api/movies – pobranie repertuaru filmowego
- POST /api/movies – dodanie nowego filmu
- DELETE /api/movies/{id} – usunięcie filmu
- GET /api/reservations/occupied – pobranie zajętych miejsc
- POST /api/reservations – zapis nowej rezerwacji

- GET /api/reservations/user/{email} – pobranie rezerwacji użytkownika
- DELETE /api/reservations/{id} – usunięcie rezerwacji
- POST /api/auth/register – rejestracja użytkownika
- POST /api/auth/login – logowanie użytkownika

3.3. Autoryzacja i obsługa użytkownika

W aplikacji zaimplementowano mechanizm autoryzacji umożliwiający rejestrację nowych użytkowników oraz logowanie do systemu. Mechanizm ten pozwala na powiązanie rezerwacji z konkretnym użytkownikiem oraz umożliwia wyświetlanie listy jego rezerwacji w sekcji „Moje rezerwacje”.

Obsługa autoryzacji została podzielona na dwie części:

- operacje wykonywane po stronie backendu,
- obsługa formularzy logowania i rejestracji po stronie front-end.

Struktura danych użytkownika

Informacje o użytkownikach przechowywane są w bazie danych w tabeli users. Strukturę danych użytkownika reprezentuje encja User.

```
@Entity
@Table(name = "users")
public class User {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String email;
private String password;

}
```

Listing 3.13 Struktura encji użytkownika

Encja przechowuje podstawowe dane użytkownika: identyfikator, adres e-mail oraz hasło. Dane te wykorzystywane są podczas procesu logowania i rejestracji użytkownika.

Dostęp do danych użytkownika realizowany jest przy użyciu repozytorium JPA.

```
public interface UserRepository extends JpaRepository<User, Long> {
Optional<User> findByEmail(String email);
}
```

Listing 3.14 Repozytorium użytkowników

Repozytorium umożliwia wyszukiwanie użytkownika na podstawie adresu e-mail. Funkcja ta wykorzystywana jest podczas procesu logowania użytkownika.

Rejestracja użytkownika

Podczas rejestracji użytkownik wprowadza adres e-mail oraz hasło w formularzu rejestracyjnym.
Endpoint odpowiedzialny za rejestrację użytkownika:

POST /api/auth/register

Backend odbiera dane użytkownika i zapisuje je w bazie danych. Przed zapisaniem sprawdzane jest, czy użytkownik o podanym adresie e-mail już istnieje.

```
@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody User user) {
   if (userRepository.findByEmail(user.getEmail()).isPresent()) {
      return ResponseEntity.badRequest().body("Użytkownik już istnieje");
   }
   return ResponseEntity.ok(userRepository.save(user));
}
```

Listing 3.15 Rejestracja użytkownika w AuthController

Jeżeli adres e-mail znajduje się już w bazie danych, backend zwraca odpowiedź błędu. W przeciwnym przypadku tworzony jest nowy rekord użytkownika.

Logowanie użytkownika

Logowanie użytkownika polega na przesłaniu danych logowania do backendu i weryfikacji ich poprawności.

Endpoint logowania:

POST /api/auth/login

Backend wyszukuje użytkownika o podanym adresie e-mail i porównuje zapisane hasło z hasłem przesłanym w zapytaniu.

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody User user) {
   Optional<User> found = userRepository.findByEmail(user.getEmail());
   if (found.isPresent() && found.get().getPassword().equals(user.getPassword())) {
      return ResponseEntity.ok(found.get());
   } else {
      return ResponseEntity.status(401).body("Nieprawidłowy email lub hasło");
   }
}
```

Listing 3.16 Logowanie użytkownika

Jeżeli dane logowania są poprawne, backend zwraca dane użytkownika. W przeciwnym przypadku zwracany jest komunikat o błędzie.

Obsługa logowania po stronie front-end

Po stronie front-end logowanie realizowane jest przy użyciu formularza logowania. Użytkownik wprowadza adres e-mail oraz hasło, które następnie są wysyłane do backendu

przy użyciu funkcji fetch[7].

```
async function login(email, password) {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!res.ok) {
    const errorText = await res.text();
    throw new Error(errorText || "Login failed");
  }
  return res.json();
}
```

Listing 3.17 Logowanie użytkownika w aplikacji front-end

Funkcja wysyła zapytanie HTTP typu POST do backendu i oczekuje odpowiedzi zawierającej dane użytkownika. W przypadku błędu zwracany jest komunikat informujący o niepoprawnych danych logowania.

Zapisywanie danych użytkownika w przeglądarce

Po poprawnym zalogowaniu aplikacja zapisuje adres e-mail użytkownika w pamięci przeglądarki przy użyciu mechanizmu localStorage.

```
localStorage.setItem("email", email);
window.location.href = "/";
```

Listing 3.18 Zapisywanie danych użytkownika

Dzięki temu aplikacja może w kolejnych widokach korzystać z informacji o aktualnie zalogowanym użytkowniku.

Wykorzystanie danych użytkownika w systemie rezerwacji

Adres e-mail zapisany w pamięci przeglądarki wykorzystywany jest podczas tworzenia rezerwacji. Dane te są przekazywane do backendu jako element obiektu rezerwacji.

Przykładowa struktura danych wysyłanych do backendu:

```
{
  "movieId": 1,
  "time": "18:00",
  "userEmail": "admin@gmail.com",
  "seats": [5, 6]
}
```

[7] Fetch API – MDN Web Docs
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Dzięki temu każda rezerwacja jest powiązana z konkretnym użytkownikiem.

Ograniczenia obecnej implementacji

Zaimplementowany system autoryzacji ma charakter uproszczony. Hasła użytkowników przechowywane są w bazie danych w formie tekstowej i nie zastosowano mechanizmu tokenów JWT ani systemu sesji.

W systemach produkcyjnych stosuje się zazwyczaj dodatkowe zabezpieczenia, takie jak szyfrowanie haseł oraz autoryzację opartą o tokeny.

Pomimo tych uproszczeń zastosowane rozwiązanie pozwala poprawnie realizować funkcje logowania, rejestracji oraz przypisywania rezerwacji do użytkownika

3.4. Testowanie aplikacji

Po zakończeniu implementacji aplikacji przeprowadzono testy systemu w środowisku lokalnym. Testowanie miało na celu sprawdzenie poprawności działania najważniejszych funkcjonalności aplikacji oraz weryfikację komunikacji pomiędzy frontendem, backendem i bazą danych.

W projekcie zastosowano dwa typy testów:

- testowanie endpointów REST API,
- testowanie funkcjonalne interfejsu użytkownika.

Testy endpointów przeprowadzono przy użyciu narzędzia **Thunder Client** [8]dostępnego w środowisku **Visual Studio Code**. Narzędzie to umożliwia wysyłanie zapytań HTTP do backendu oraz analizę odpowiedzi serwera.

Testowanie funkcjonalne polegało natomiast na sprawdzeniu działania aplikacji z perspektywy użytkownika, czyli rejestracji, logowania, wyboru filmu, wyboru miejsc oraz zapisu rezerwacji.

Testowanie endpointów REST API

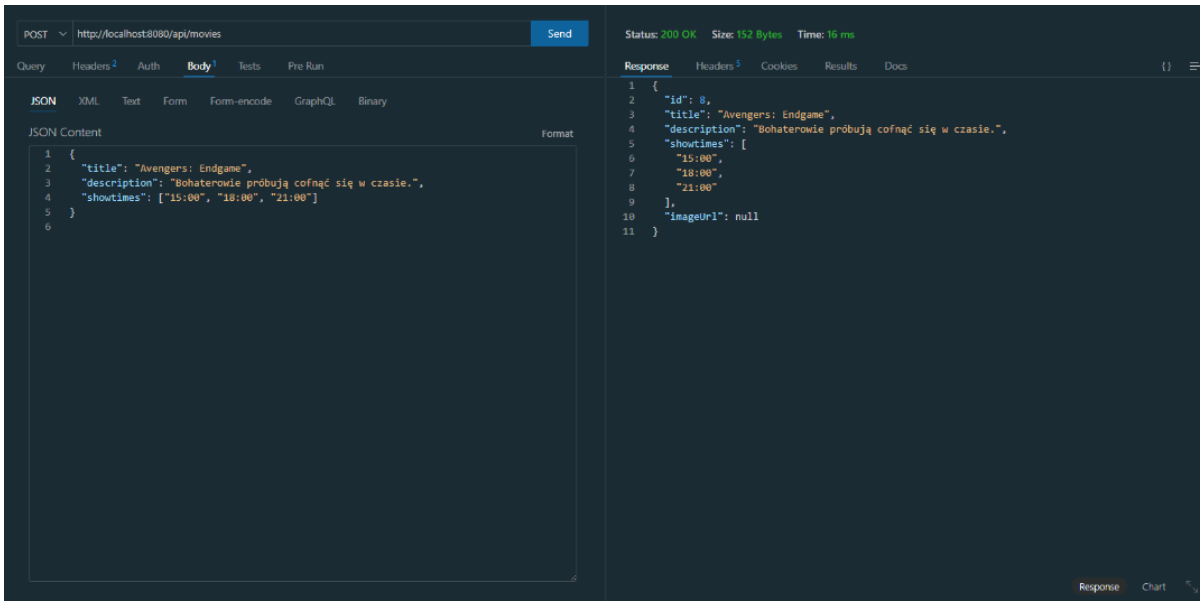
W pierwszym etapie przeprowadzono testy wybranych endpointów backendu. Celem tej części było sprawdzenie, czy serwer poprawnie odbiera dane wejściowe, zapisuje je w bazie danych oraz zwraca prawidłową odpowiedź.

Test 1. Dodanie nowego filmu

W celu sprawdzenia poprawności działania panelu administracyjnego oraz backendu przetestowano endpoint odpowiedzialny za dodawanie filmu:
POST /api/movies

Do serwera przesłano dane filmu w formacie JSON zawierające tytuł, opis oraz listę godzin seansów. Backend poprawnie zapisał nowy rekord i zwrócił odpowiedź HTTP 200.

[8] https://www.thunderclient.com/



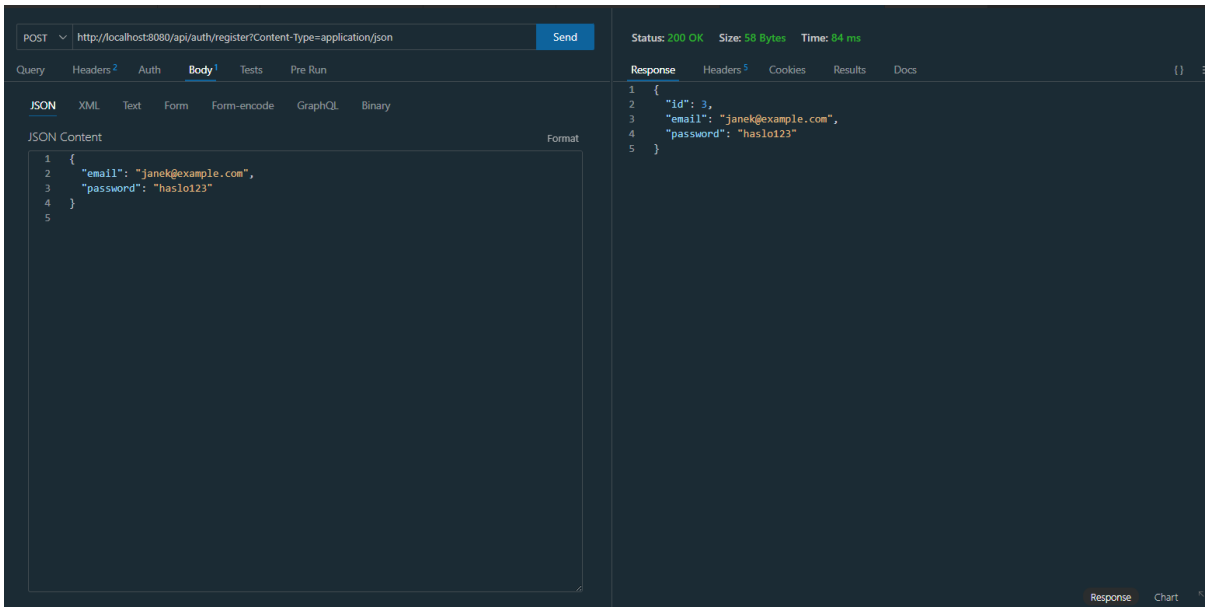
Rysunek 3.1 Test endpointu POST /api/movies w Thunder Client
Źródło: opracowanie własne

Test 2. Rejestracja użytkownika

Kolejnym etapem było sprawdzenie endpointu odpowiedzialnego za rejestrację użytkownika:

POST /api/auth/register

Do serwera przesłano adres e-mail oraz hasło nowego użytkownika. Backend utworzył rekord w tabeli users i zwrócił odpowiedź potwierdzającą poprawne wykonanie operacji.



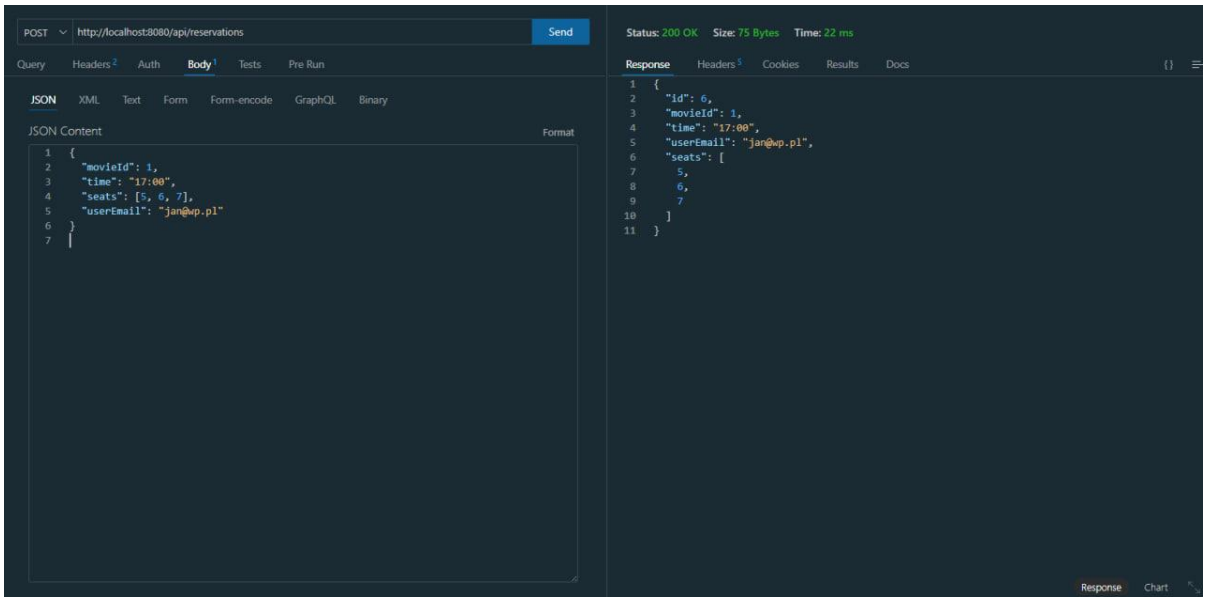
Rysunek 3.2 Test endpointu POST /api/auth/register w Thunder Client
Źródło: opracowanie własne

Test 3. Zapis nowej rezerwacji

Następnie przetestowano endpoint odpowiedzialny za zapis rezerwacji:

POST /api/reservations

Do backendu przesłano dane zawierające identyfikator filmu, godzinę seansu, adres e-mail użytkownika oraz listę wybranych miejsc. Po poprawnym przetworzeniu danych backend zwrócił odpowiedź HTTP 200 oraz dane zapisanej rezerwacji.



Rysunek 3.3 Test endpointu POST /api/reservations w Thunder Client
Źródło: opracowanie własne

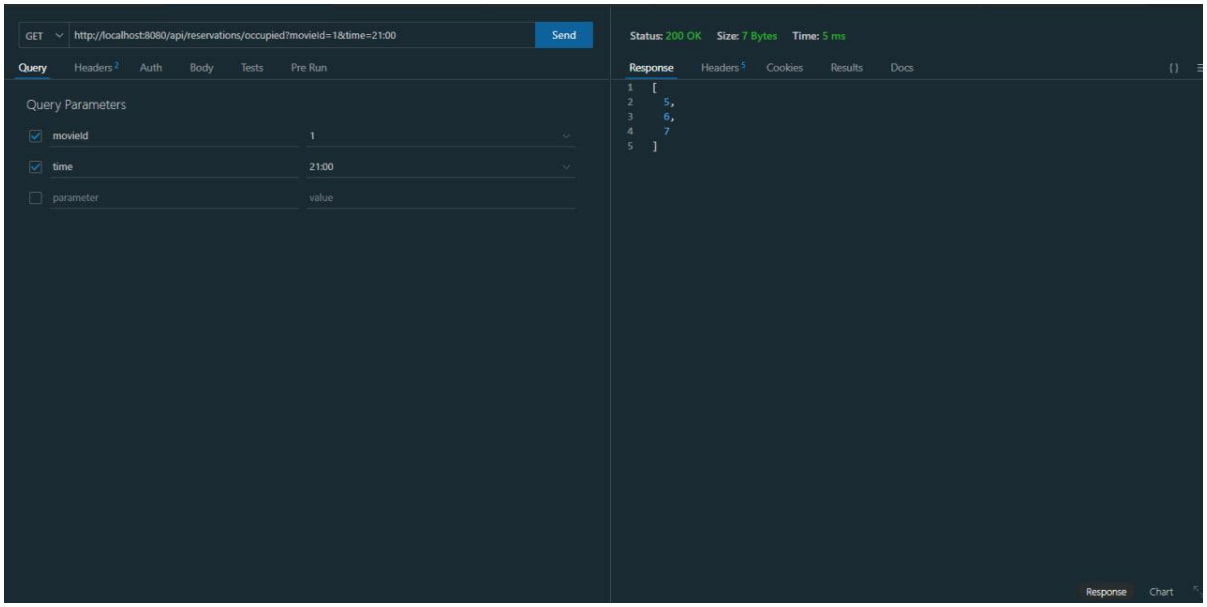
Test 4. Pobranie listy zajętych miejsc

W celu sprawdzenia poprawności działania mechanizmu rezerwacji miejsc przetestowano endpoint odpowiedzialny za pobieranie listy zajętych miejsc dla wybranego seansu.

GET /api/reservations/occupied

Do zapytania przekazano dwa parametry identyfikujące seans:

- movieId – identyfikator filmu
- time – godzina seansu



Rysunek 3.4 Test endpointu GET /api/reservations/occupied w Thunder Client
Źródło: opracowanie własne

W trakcie testów endpointów potwierdzono, że backend poprawnie obsługuje podstawowe operacje systemu: zapis filmu, rejestrację użytkownika, tworzenie rezerwacji oraz pobranie listy zajętych miejsc.

Testowanie funkcjonalne interfejsu użytkownika

Po zakończeniu testów endpointów przeprowadzono testy funkcjonalne aplikacji z perspektywy użytkownika. Celem tej części było sprawdzenie, czy interfejs działa poprawnie oraz czy użytkownik może przejść przez pełną ścieżkę korzystania z systemu.

Sprawdzono następujące scenariusze:

1. wejście na stronę główną aplikacji,
2. przejście do listy filmów,
3. wybór filmu oraz godziny seansu,
4. wyświetlenie siatki miejsc,
5. wybór wolnych miejsc,
6. zapis rezerwacji,
7. wyświetlenie listy „Moje rezerwacje”,
8. logowanie oraz rejestrację użytkownika,
9. usuwanie zapisanej rezerwacji.

Podczas testów zweryfikowano również poprawność aktualizacji interfejsu po zmianie stanu aplikacji. Po wybraniu miejsca jego wygląd zmieniał się natychmiast, a po zapisaniu rezerwacji użytkownik otrzymywał komunikat o powodzeniu operacji oraz był przekierowywany do widoku swoich rezerwacji.

Szczególną uwagę zwrócono na poprawność synchronizacji danych o zajętych miejscach. Po wejściu na stronę rezerwacji aplikacja pobierała aktualną listę miejsc zapisanych w bazie i blokowała możliwość ich ponownego wyboru.

Wyniki testów i wnioski

Przeprowadzone testy potwierdziły poprawne działanie najważniejszych funkcji systemu. Zarówno warstwa backendowa, jak i warstwa front-end poprawnie realizowały swoje zadania.

Testy endpointów REST API wykazały, że backend poprawnie:

- odbiera dane wejściowe,
- zapisuje rekordy w bazie danych,
- zwraca odpowiedzi w formacie JSON,
- obsługuje podstawowe operacje użytkownika i administratora.

Testy funkcjonalne potwierdziły natomiast, że użytkownik może przejść pełną ścieżkę działania aplikacji: od rejestracji i logowania, przez wybór filmu i miejsc, aż do zapisania rezerwacji oraz wyświetlenia jej na liście własnych rezerwacji.

Na podstawie przeprowadzonych testów można stwierdzić, że aplikacja spełnia założenia funkcjonalne przyjęte na etapie projektowania.

ZAKOŃCZENIE

Celem niniejszej pracy było zaprojektowanie oraz implementacja aplikacji webowej umożliwiającej rezerwację miejsc na seanse kinowe. W ramach realizacji projektu opracowano system działający w architekturze klient-serwer, składający się z warstwy front-end, backendu oraz relacyjnej bazy danych.

W pierwszej części pracy przeprowadzono analizę problemu rezerwacji miejsc w kinie oraz określono najważniejsze funkcjonalności systemu. Na podstawie tej analizy zidentyfikowano kluczowe wymagania aplikacji, takie jak możliwość przeglądania repertuaru filmowego, wyboru godzin seansów, wyboru miejsc na sali kinowej oraz zapisu rezerwacji w systemie.

W drugim rozdziale przedstawiono projekt systemu informatycznego. Opisano architekturę aplikacji, strukturę bazy danych oraz model danych wykorzystywany w systemie. Zaprezentowano również diagramy UML, które w graficzny sposób przedstawiają funkcjonalności systemu oraz przebieg procesu rezerwacji miejsc.

Trzeci rozdział poświęcono implementacji aplikacji. Opisano mechanizm wyboru miejsc w aplikacji front-endowej, komunikację z backendem za pomocą zapytań HTTP oraz implementację endpointów REST API odpowiedzialnych za obsługę filmów, rezerwacji oraz użytkowników. Przedstawiono również mechanizm autoryzacji umożliwiający rejestrację i logowanie użytkowników.

W końcowej części pracy przeprowadzono testowanie aplikacji w środowisku lokalnym. Testy obejmowały zarówno sprawdzenie działania endpointów REST API przy użyciu narzędzia Thunder Client, jak i testy funkcjonalne interfejsu użytkownika. Wyniki testów potwierdziły poprawne działanie systemu oraz zgodność implementacji z założeniami projektowymi.

Zrealizowany projekt spełnia założone wymagania funkcjonalne i umożliwia użytkownikowi przeglądanie repertuaru filmowego, wybór miejsc na sali kinowej oraz zapisywanie rezerwacji w systemie. Dzięki zastosowaniu architektury klient–serwer oraz technologii takich jak React, Spring Boot oraz MySQL aplikacja może być w przyszłości łatwo rozwijana o kolejne funkcjonalności.

Możliwe kierunki dalszego rozwoju systemu obejmują między innymi wprowadzenie bardziej zaawansowanego mechanizmu autoryzacji użytkowników, zastosowanie szyfrowania haseł oraz rozszerzenie funkcji administracyjnych umożliwiających zarządzanie repertuarem oraz seansami filmowymi.

BIBLIOGRAFIA

[1] React – Official documentation,
https://react.dev

[2] Spring Boot – Official documentation,
https://spring.io/projects/spring-boot

[3] Fielding R., REST API architectural style,
https://restfulapi.net/

[4] MySQL Reference Manual,
https://dev.mysql.com/doc/

[5] JSON format specification,
https://www.json.org

[6] Spring Data JPA documentation,
https://spring.io/projects/spring-data-jpa

[7] Fetch API – MDN Web Docs,
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[8] Thunder Client,
https://www.thunderclient.com/

AKTY NORMATYWNE

1. **Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 z dnia 27 kwietnia 2016 r. w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych i w sprawie swobodnego przepływu takich danych (RODO).**

2. **Ustawa z dnia 18 lipca 2002 r. o świadczeniu usług drogą elektroniczną (Dz.U. 2002 nr 144 poz. 1204 z późn. zm.).**

3. **Ustawa z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz.U. 1994 nr 24 poz. 83 z późn. zm.).**

4. **Ustawa z dnia 10 maja 2018 r. o ochronie danych osobowych (Dz.U. 2018 poz. 1000).**